EXP-5

Student Name: ANSHAJ CHAUHAN **UID:** 24BAI70503
Branch: AIT-CSE (AIML) **Section/Group:** 24AIT_KRG-1A
Semester: 5 **Subject Code:** 24CSP-310
Subject Name : Advanced Database Management System

A)

(A) A website maintains records of users who have registered on the platform and the status of confirmation requests sent to them. A confirmation request can either be successfully completed or may expire without completion.
For each registered user, calculate the confirmation rate, which is defined as:

```
Confirmation Rate = Number of Successful Confirmations
                    ----------------------------------
                    Total Confirmation Requests
```

If a user has never received any confirmation request, their confirmation rate should be considered **0**.

Display the user ID along with the confirmation rate rounded to **2 decimal places**. The result can be returned in any order.

CODE:

```sql
CREATE TABLE Signups
(
    user_id INT PRIMARY KEY,
    time_stamp TIMESTAMP
);

CREATE TABLE Confirmations
(
    user_id INT,
    time_stamp TIMESTAMP,
    action VARCHAR(20),
    PRIMARY KEY(user_id, time_stamp),
    FOREIGN KEY(user_id)
    REFERENCES Signups(user_id)
);
```

```sql
INSERT INTO Signups
VALUES
(3,'2020-03-21 10:16:13'),
(7,'2020-01-04 13:57:59'),
(2,'2020-07-29 23:09:44'),
(6,'2020-12-09 10:39:37');


INSERT INTO Confirmations
VALUES
(3,'2021-01-06 03:30:46','timeout'),
(3,'2021-07-14 14:00:00','timeout'),
(7,'2021-06-12 11:57:29','confirmed'),
(7,'2021-06-13 12:58:28','confirmed'),
(7,'2021-06-14 13:59:27','confirmed'),
(2,'2021-01-22 00:00:00','confirmed'),
(2,'2021-02-28 23:59:59','timeout');
```

```sql
SELECT
    S.user_id,
    ROUND(
    COALESCE(
        SUM(CASE WHEN C.action = 'confirmed' THEN 1 ELSE 0 END) * 1.0
        / NULLIF(COUNT(C.user_id), 0),
        0
    ),
    2
) AS confirmation_rate
FROM Signups S
LEFT JOIN Confirmations C
    ON S.user_id = C.user_id
GROUP BY S.user_id;
```

OUTPUT SCREENSHOTS:

	user_id [PK] integer	confirmation_rate numeric
1	2	0.50
2	6	0.00
3	3	0.00
4	7	1.00

B)

(B) Problem Statement
A ride-booking company maintains information about trip requests and the users involved in those trips. Some users may be restricted from using the platform, while others are active participants. Each trip can either be completed successfully or cancelled by one of the participants.

For each day between **2013-10-01** and **2013-10-03**, determine the proportion of canceled trips among all valid trips. A trip should be considered valid only when both the customer and the driver associated with that trip are active users. The final result should display the date and the calculated cancellation rate rounded to two decimal places.

We need to calculate:

Cancellation Rate

=

Cancelled Trips

Total Valid Trips

But only consider trips where:

Client is NOT banned
AND
Driver is NOT banned

and

Date between

2013-10-01
And
2013-10-03

CODE:

```
CREATE TABLE Users
(
    users_id INT PRIMARY KEY,
    banned VARCHAR(3),
    role VARCHAR(20)
);

CREATE TABLE Trips
(
    id INT PRIMARY KEY,
    client_id INT,
    driver_id INT,
    city_id INT,
    status VARCHAR(30),
    request_at DATE
);
```

```sql
INSERT INTO Users
VALUES
(1,'No','client'),
(2,'Yes','client'),
(3,'No','client'),
(4,'No','client'),
(10,'No','driver'),
(11,'No','driver'),
(12,'No','driver'),
(13,'No','driver');

INSERT INTO Trips
VALUES
(1,1,10,1,'completed','2013-10-01'),
(2,2,11,1,'cancelled_by_driver','2013-10-01'),
(3,3,12,6,'completed','2013-10-01'),
(4,4,13,6,'cancelled_by_client','2013-10-01'),
(5,1,10,1,'completed','2013-10-02'),
(6,2,11,6,'completed','2013-10-02'),
(7,3,12,6,'completed','2013-10-02'),
(8,2,12,12,'completed','2013-10-03'),
(9,3,10,12,'completed','2013-10-03'),
(10,4,13,12,'cancelled_by_driver','2013-10-03');
```

```sql
SELECT
    T.request_at AS Day,
    ROUND(
        SUM(
            CASE
                WHEN T.status != 'completed' THEN 1
                ELSE 0
            END
        )::NUMERIC / COUNT(*),
        2
    ) AS "Cancellation Rate"
FROM Trips T
JOIN Users C
    ON T.client_id = C.users_id
JOIN Users D
    ON T.driver_id = D.users_id
WHERE C.banned = 'No'
  AND D.banned = 'No'
  AND T.request_at BETWEEN '2013-10-01' AND '2013-10-03'
GROUP BY T.request_at
ORDER BY T.request_at;
```

OUTPUT SCREENSHOTS:

	day date	Cancellation Rate numeric
1	2013-10-01	0.33
2	2013-10-02	0.00
3	2013-10-03	0.50

LEARNING OUTCOMES:

- Learned to perform how to use join multiple times
- Gained experience in using self-joins .
- Learned to use round aggregate function.
- Learned how to use case condition.
- Improved proficiency in writing optimized SQL queries for real-world scenarios.

CONCLUSION:

Successfully implemented advanced SQL queries involving custom sorting, self-joins, conditional aggregation, and percentage calculations to produce accurate results.